

Experiment 1: Case Study

KL ERP System — Integrated Academic & Administrative Management Platform

Atharva Jain (2420030092)

Problem Statement: Choose a real-world case study, create user stories, select an appropriate software development process model, identify a suitable programming language and technologies, and propose an implementation and evaluation strategy supported by UML diagrams.

Real-World Case Study: KL ERP System — an integrated academic and administrative management platform connecting students, faculty, and administrators.

1. Introduction

What is the KL ERP System?

The KL ERP System is an integrated academic and administrative management platform used across an educational institution. It brings course registration, attendance, examinations, fee payment, and timetabling into a single digital system, replacing scattered registers and manual processes with one connected source of truth for students, faculty, and administrators.

Core idea:

One platform, three connected roles, real-time academic and administrative record-keeping from enrollment to graduation.

Scope of the System

- Student registration, login, and profile management
- Course registration and timetable viewing
- Attendance recording and tracking
- Examination result publishing
- Online fee payment
- Course catalog and timetable configuration
- Institutional reporting and communication

Functional Requirements

Student

- Log in securely and register for courses
- View attendance and examination results
- Pay fees online and view timetable

Faculty

- Record attendance and enter marks
- Upload course materials
- View assigned courses and class schedules

Administrator

- Manage student records and fee structure
- Manage course catalog and configure timetables

- Generate institutional reports and send communications

2. User Stories

A User Story is a short, simple description of a software feature written from the end user's perspective, in the format: "As a [user], I want [functionality] so that [benefit]."

Student

User ID	User Story	Priority
US-01	As a student, I want to log in securely so that I can access my academic information.	High
US-02	As a student, I want to register for courses online so that I can select my subjects easily.	High
US-03	As a student, I want to view my attendance so that I can monitor my academic progress.	Medium
US-04	As a student, I want to check examination results so that I can evaluate my performance.	High
US-05	As a student, I want to pay fees online so that I can complete payments conveniently.	High
US-06	As a student, I want to view my timetable so that I know my class schedule.	Medium

Faculty

User ID	User Story	Priority
US-07	As a faculty member, I want to record attendance so that student attendance can be tracked accurately.	High
US-08	As a faculty member, I want to upload course materials so that students can access learning resources.	Medium
US-09	As a faculty member, I want to enter marks so that students can view their academic performance.	High
US-10	As a faculty member, I want to view my class schedule so that I can plan my teaching week.	Low

Administrator

User ID	User Story	Priority
US-11	As an administrator, I want to manage student records so that all student information remains updated.	High
US-12	As an administrator, I want to manage the fee structure so that payments are processed correctly.	Medium
US-13	As an administrator, I want to manage the course catalog so that offerings stay current each semester.	High

User ID	User Story	Priority
US-14	As an administrator, I want to configure timetables so that classes are scheduled without conflicts.	Medium
US-15	As an administrator, I want to generate reports so that institutional performance can be monitored.	Medium

3. Software Development Process Model

Agile Scrum

Reason for Selection

- Requirements may change frequently across departments (academic, finance, admin).
- Different departments give continuous feedback throughout development.
- Modules such as attendance, fees, and examinations can be developed incrementally.
- Faster delivery and testing of individual features.
- Easier to isolate and fix defects within a sprint.
- Well suited for large, multi-module institutional web applications.

Sprint Plan

Sprint	Features
Sprint 1	User Registration & Secure Login
Sprint 2	Course Registration Module
Sprint 3	Attendance Management
Sprint 4	Examination & Results Module
Sprint 5	Fee Payment Gateway
Sprint 6	Timetable & Communication Module
Sprint 7	Admin Dashboard & Reports
Sprint 8	Testing & Deployment

4. Programming Language and Technologies

Backend

Java (Spring Boot)

- Secure and platform-independent
- Scalable for institution-wide, multi-department use
- Well suited for enterprise-level applications
- Strong REST API support
- Excellent database connectivity
- Large community and long-term support

Frontend

- HTML5
- CSS3
- JavaScript
- React.js

Database

- MySQL

Other Technologies

- REST API
- JWT Authentication
- Payment Gateway Integration (Razorpay/Stripe)
- Git / GitHub
- Docker
- Cloud Hosting (AWS)

5. Implementation Strategy

Phase 1 – Requirement Analysis

- Gather requirements from students, faculty, and administrators.
- Create Software Requirement Specification (SRS).
- Prepare use case diagrams.

Phase 2 – System Design

- Design database schema.
- Design UI wireframes.
- Create UML diagrams.
- Develop system architecture.

Phase 3 – Development

- Implement user authentication.
- Develop course registration module.
- Build attendance and examination modules.
- Integrate fee payment gateway.
- Build admin dashboard and reporting module.

Phase 4 – Testing

Unit Testing

- Login
- Course registration
- Fee payment

Integration Testing

- Attendance + Examination results
- Fee payment + Student records

System Testing

- Complete application testing

User Acceptance Testing (UAT)

- Verify institutional business requirements.

Phase 5 – Deployment

- Docker Container
- AWS EC2
- MySQL Database
- Nginx Web Server

Phase 6 – Maintenance

- Fix defects, upgrade features, and improve performance and security.

6. Evaluation Strategy

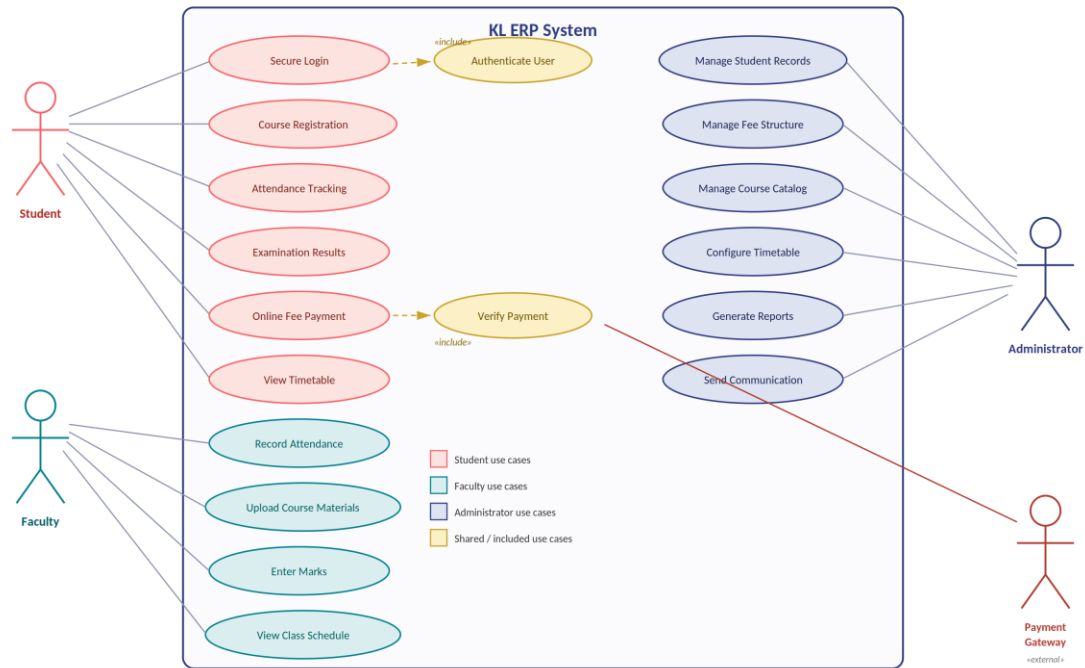
Evaluation Parameter	Method
Functional Accuracy	Requirement verification against user stories
Performance	Load testing using JMeter
Security	Authentication and penetration testing
Reliability	Continuous monitoring, backup and recovery checks
Usability	User feedback survey (students, faculty, admin)
Scalability	Concurrent user testing
Maintainability	Code review and documentation

Expected Outcomes

- Centralized platform for all academic and administrative operations.
- Improved communication between students, faculty, and administration.
- Reduced paperwork and fewer manual errors.
- Faster processing of academic activities.
- Secure and reliable online fee payments.
- Accurate, real-time attendance and examination records.
- Scalable architecture that can grow with the institution.

7. UML Diagrams

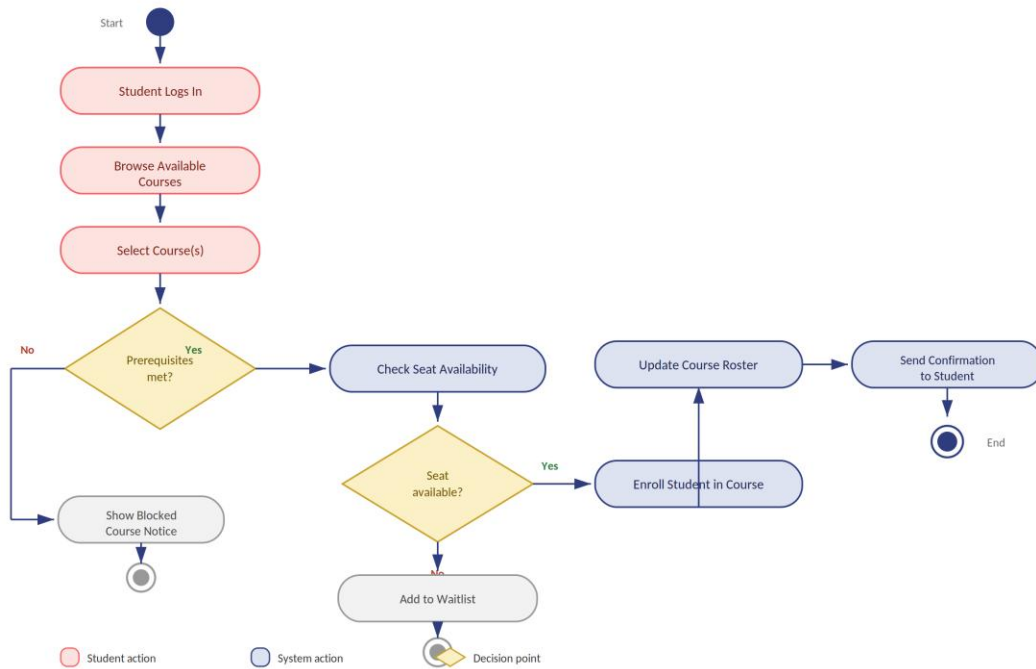
7.1 Use Case Diagram



The diagram shows the three actors of the KL ERP System — Student, Faculty, and Administrator — along with Payment Gateway as an external system actor. “Authenticate User” and “Verify Payment” are shared use cases pulled in through «include» relationships, since login is a prerequisite for role-specific actions and payment verification is required before a fee payment is confirmed.

7.2 Activity Diagram — Course Registration Flow

Activity Diagram — Course Registration Flow



The activity diagram traces a student's course registration from login to confirmation. It includes two decision points: whether course prerequisites are met (ending with a blocked-course notice if not) and whether a seat is available (adding the student to a waitlist if not). On the successful path, the system enrolls the student, updates the course roster, and sends a confirmation.

8. Conclusion

The KL ERP System brings academic and administrative operations onto a single connected platform, giving students, faculty, and administrators one consistent, real-time source of institutional data.

User Stories

Captured the real needs of students, faculty, and administrators, forming a clear and testable basis for system requirements.

Agile Scrum

Enabled incremental delivery and continuous departmental feedback across eight focused sprints, suited to a large, multi-module institutional system.

Java / React / MySQL Stack

Provided a secure, scalable, and maintainable foundation capable of handling institution-wide academic and administrative workloads.

Structured Strategy

Ensured systematic design, development, testing, and evaluation, resulting in a reliable platform ready for real-world institutional deployment.